

Pre-Kickoff Voice Agent — Full Project Plan

Rocketlane PS/Onboarding/Implementation Hackathon · June 18, 12:30–6:00 PM · 5 hours · No-code

Build with: Claude (Anthropic) · ElevenLabs · Lovable · Nitro by Rocketlane

0. The wedge (read this first)

Every AI tool in the PS/onboarding space today works on **artifacts that already exist** — it ingests SOWs, emails, recorded calls, and configs that someone already produced, and rearranges them. Rocketlane's own agent **Nitro** already does this extremely well: it picks up SOWs, PDFs, emails, and call recordings and auto-generates project plans and documents. The market for "parse the documents you already have" is crowded and, for this audience, already owned by the people judging us.

Our wedge is the one input nobody captures: the conversations that should happen before kickoff but never do — run by a voice agent, not a form, not a document parser.

After a deal closes, onboarding burns weeks of human time, and a large share of it is spent on **calls** — requirements gathering, billing/legal detail collection, scope confirmation, scheduling, status check-ins. A lot of those calls don't need a human's judgment; the human is just acting as a microphone and a transcription service. Our product puts a **voice agent** on exactly those calls. It speaks to each stakeholder directly, captures what no document contains — the sponsor's unspoken worry, the IT lead's "that timeline is fantasy," the champion's adoption fear — and turns every conversation into structured paperwork and a **cross-stakeholder conflict map** automatically.

Two things make this defensible:

1. **It's voice-first and outbound.** It generates *net-new* context by talking to people, rather than re-processing context that already exists in a file. That's the gap in the market.
2. **It produces an artifact no document tool can: the conflict map.** Interviewing four stakeholders separately and surfacing where they silently contradict each other (sponsor says 6 weeks, IT says Q2) is impossible to derive from a SOW. This is the alignment layer Nitro can't generate because the data isn't in any document — it only exists in people's heads until someone asks.

Framing for the room: we are not competing with Nitro — we are the missing **input layer** that feeds it. Nitro turns documents into plans; we turn *conversations* into the structured human context that makes those plans actually right. We make their agent smarter by giving it the data it can't get on its own.

1. One-line pitch

A voice agent that takes the onboarding calls a human shouldn't have to — interviewing every stakeholder before kickoff, generating the paperwork automatically, and surfacing the misalignments no SOW can reveal.

2. The problem

When a deal closes, two companies enter a weeks-long onboarding process that eats enormous human resource. Strip it down and the work splits into two kinds of calls:

- **Judgment calls** — negotiation, relationship-building, executive trust, tense scope conversations. These genuinely need a human and always will.
- **Information calls** — requirements intake, billing/legal detail collection, scope confirmation, scheduling, routine status updates. Here the human isn't adding judgment; they're a conduit moving information from the customer's head into a document. These are most of the calls.

Today, a human runs *both* kinds, and then does the paperwork afterward by hand. The information calls are pure overhead — expensive people spent as transcription machines — and the paperwork they generate is slow, inconsistent, and error-prone.

On top of the call overhead, the surrounding admin is a scramble of manual work across email, spreadsheets, and Slack: contracts routed by hand, billing details collected from whoever answers first, compliance questionnaires lost and re-sent, stakeholders looped in late. And the kickoff call itself becomes a disguised discovery call, because nobody had time to prep.

Two costs:

Time. A PM loses 5–10 hours of pre-project work per engagement before real delivery begins — much of it sitting on information calls and writing them up. Across a portfolio, that's weeks per quarter of senior-person time.

Error and misalignment. Manual paperwork produces wrong billing entities, missed DPAs, signatory mismatches. Worse, because each stakeholder is interviewed (if at all) in isolation and never cross-checked, the project starts with hidden disagreements — sponsor and IT have different timelines in their heads, and nobody finds out until it's a blocker three sprints in.

The thesis: put a voice agent on the information calls. Let it talk to each stakeholder, generate the paperwork, and — because it's the only thing that has spoken to *everyone* — surface the conflicts before day one. Humans keep the judgment calls. The agent takes the rest and does the write-up.

3. The product

A **voice-first pre-kickoff agent** with a document-processing layer underneath it. Voice is the MVP and the differentiator; document automation is the supporting engine that makes the voice output actionable and feeds Rocketlane.

Two halves, clearly ranked

THE WEDGE (primary — this is what we demo and lead with): Voice stakeholder intake → conflict map → paperwork. The agent runs the information calls. It interviews each stakeholder by voice, adapts its questions live, and produces three things no existing tool produces together: a per-stakeholder summary, the auto-generated onboarding paperwork from what was said, and the **cross-stakeholder conflict map**.

THE ENGINE (supporting — table stakes, and where we connect to Nitro): Document ingestion + auto-fill. The agent reads the signed SOW/MSA once to build a structured **Deal Profile**, then uses it to (a) know who to call and what to ask, and (b) auto-fill downstream documents and flag inconsistencies. We are explicit that this half overlaps with what Nitro already does — so we position it as the bridge: our voice layer produces context, the engine structures it, and it all lands in Rocketlane as a pre-built project.

4. Where the voice agent operates (the call-triage map)

This is the heart of the wedge and the thing that signals we understand the actual work. Every onboarding touchpoint is sorted into who should run it:

Touchpoint	Today	Our model	Why
Requirements / discovery intake	Human call + manual write-up	Voice agent	Information call — agent gathers and documents
Billing & legal detail collection	Email chase / human call	Voice agent	Pure data collection from finance contact
Scope confirmation	Human call	Voice agent	Read-back and confirm; flag disputes for human
Compliance/security questionnaire walkthrough	Human call / form	Voice agent	Structured Q&A, routes blockers
Scheduling & logistics	Human / back-and-forth email	Voice agent	Trivially automatable
Routine status check-ins	Human call	Voice agent	Templated; agent captures and reports
Kickoff prep / stakeholder context	Skipped (no time)	Voice agent	The discovery that never happens today
Contract negotiation	Human	Human	Judgment call — agent stays out
Relationship / trust building	Human	Human	Judgment call
Escalations & conflict resolution	Human	Human	Judgment call

The pitch line: "The agent takes the calls that don't need a human. The human keeps the calls that do. Most onboarding calls are the first kind."

Every cell marked "Voice agent" is a place we can show ElevenLabs + Claude running an actual conversation and producing an actual document. That's the demo surface, and it's broad.

5. The conflict map — our signature artifact

This is the single most defensible thing we build, because it cannot be produced from documents and no competitor generates it.

The agent is the only entity that has spoken privately to every stakeholder. After all calls, a Claude synthesis pass looks across the transcripts for contradictions the stakeholders don't know they have:

- **Timeline conflicts** — sponsor's "live in 6 weeks" vs. IT's "realistically Q2"
- **Success-criteria drift** — every stakeholder defines "done" differently
- **Authority gaps** — decisions everyone assumes someone else owns
- **Unspoken assumptions** — "we'll just integrate with Salesforce," casual from the sponsor, dreaded by IT
- **Political/emotional signals** — hesitation, frustration, things said carefully (only audible in *voice*, invisible in a form)

Output: a one-screen map, conflicts in red, each with the two contradicting quotes and a suggested resolution for the kickoff agenda. *This is the thing that makes a judge go "oh."* It's also the thing that proves voice matters — half these signals don't survive being typed into a form.

6. The supporting engine: Deal Profile & document auto-fill

The voice layer needs to know who to call and what to ask; the paperwork needs structured fields. Both come from reading the deal once.

The Deal Profile (structured source of truth)

Built from the SOW/MSA via Claude, enriched, and completed by asking a human *once* for anything missing. Fields:

- **Customer/entity:** legal entity, billing entity (often different), addresses, tax ID, industry, size, region
- **Commercial:** total value, currency, payment terms, schedule/milestones, PO requirement, term length (often *derived* from dates)
- **Scope:** summary, deliverables, exclusions, start date, go-live, milestones, dependencies
- **People:** signatories, exec sponsor, project lead, IT/technical, finance, end-user champion, procurement (names, emails, roles) — *this list is who the voice agent calls*
- **Compliance:** required docs (derived), special clauses flagged, data residency, security review level
- **Meta:** confidence score per field, source per field, fields flagged for human review

Document auto-fill (four fill types)

Fill type	Definition	Example
Direct	Copied from a profile field	Billing entity → order form
Derived	Computed from fields	Term length from start + end dates
Conditional	Included only if a condition holds	GDPR addendum only if region = EU
Flagged blank	Can't be confidently filled → marked for the one human review pass, never guessed	A signatory title stated nowhere

Required-documents matrix (conditional logic)

Condition	Document triggered
All deals	Order form, billing setup, kickoff template
Enterprise	SOC 2 request, security questionnaire
Any data processing	DPA
Region = EU/EEA	GDPR addendum, data residency confirmation
Industry = healthcare	BAA
US / non-US customer	W-9 / W-8

Consistency check (error reduction)

After fill, cross-check documents against each other: billing entity identical across order form, DPA, billing setup? Signatory on DPA matches MSA? Dates and currency consistent? Cross-document mismatches are where expensive human errors hide; the system catches them in seconds.

Honest positioning: this engine overlaps with Nitro. We say so on purpose. Our claim isn't "we built a better document parser in 5 hours" — it's "our voice layer produces the human context Nitro can't capture, and the engine structures it into Rocketlane." The engine is the bridge, not the headline.

7. End-to-end flow

1. Deal closes. PM drops the signed SOW/MSA into the tool.
2. Claude builds the Deal Profile — terms, scope, **and the stakeholder list to call** — with confidence scores and flagged fields.
3. Required-documents matrix fires; the document set assembles and auto-fills from the profile; consistency check runs.
4. **The voice agent goes to work.** Scheduling links go out; the agent calls each stakeholder for their information call, adapting questions by role and to what they say.
5. Each call auto-generates its paperwork (requirements captured, billing collected, scope confirmed) and stores the transcript.
6. After all calls, Claude produces the **conflict map** and the kickoff packet.
7. PM does **one** review pass — confirms flagged fields, reads the conflicts, approves.
8. Output lands in Rocketlane as a pre-built project: stakeholders as contacts, risks/conflicts as register entries, agenda as the kickoff template, success criteria as goals.
9. Day one: contracts in motion, billing set up, stakeholders mapped, conflicts surfaced, agenda drafted — and the humans only spent time on the judgment calls.

8. Architecture & tooling

We use a **hybrid build**: Lovable for the frontend (fast, sponsor, no-code), then export to GitHub and finish the DB, backend, and integrations in Cursor/Claude Code (control over the hard parts).

Layer	Tool	Role
Frontend (the shell)	Lovable	Project setup, SOW upload, Deal Profile viewer, document viewer, conflict map + packet display, review UI. Built first, exported early.
DB + backend + integrations	Cursor / Claude Code	Pull the exported repo; build the DB logic, backend functions, and the Anthropic + ElevenLabs wiring here
Database	Supabase (Postgres)	Keep what Lovable provisions; seeded dummy data
Voice (the wedge)	ElevenLabs Conversational AI	Outbound stakeholder calls, natural voice — no telephony to build
Reasoning	Claude (Anthropic)	SOW → Deal Profile; live question generation mid-call; cross-transcript conflict synthesis; document fill
Destination	Nitro / Rocketlane	Output as a pre-built Rocketlane project (mocked) — we are the input layer feeding it

Why this hybrid (settled): Lovable is a sponsor, the event is no-code-friendly, and it builds the demo-facing UI fast — but the ElevenLabs voice wiring and the Anthropic calls want direct code control, which is where Cursor/Claude Code is stronger. So: **Lovable frontend → export to GitHub early → Cursor for everything else.** Order matters — Lovable scaffolds the whole project skeleton, so let it lay that foundation, export once, and from then on Cursor is the single source of truth (going backend-first and importing the frontend later means a messy merge). Keep the Supabase DB Lovable provisions rather than re-rolling one. Budget ~15-30 min for the export/handoff — it's real time in a 5-hour window. Using all three sponsor tools (Lovable + ElevenLabs + Claude) is also strategically good for judging.

9. Scope for 5 hours — build vs. mock

Build the voice wedge for real; mock the parts that overlap with Nitro.

Build for real: 1. **SOW → Deal Profile** (the core extraction; feeds everything) 2. **Voice stakeholder intake** — at least one live role (sponsor) end-to-end via ElevenLabs, plus one pre-recorded second stakeholder 3. **Conflict map + kickoff packet** — the signature artifact (spend the most time here) 4. **One call → one document** auto-generated live, to prove "calls become paperwork"

Mock in the UI (show, don't build): - Full document auto-fill across the whole matrix (show one filled doc + the matrix firing) - Compliance routing and signature chasing (show "sent"/tracked states) - Rocketlane export (show the populated-project preview)

This yields a demo that *feels* like the whole product while the truly-live pieces are the voice call, the generated paperwork, and the conflict map — i.e. the wedge.

10. Hour-by-hour build plan

Phasing follows the hybrid flow: **Lovable frontend → export → Cursor for everything else**, and inside the Cursor phase the **conflict map is built before voice** (it's the differentiator; voice is the riskiest integration).

Time	Phase	Milestone
0:00-0:30	Setup	Tools connected, roles assigned, sample SOW + two planted-conflict transcripts prepared, demo arc agreed
0:30-1:15	Lovable (frontend)	Build the screens as a UI shell with mock data; conflict map screen polished
1:15-1:30	Export / handoff	Push to GitHub, pull into Cursor, run locally, wire secrets (budget the ~15-30 min)
1:30-2:10	Cursor — core engine	Claude extraction → a real SOW becomes a real Deal Profile with a stakeholder list, wired into the UI
2:10-3:05	Cursor — conflict map (the magic)	Claude synthesis across the two seeded transcripts → conflict map renders with a real contradiction in red. Built before voice on purpose.
3:05-4:05	Cursor — voice agent (the wedge)	ElevenLabs sponsor call end-to-end; transcript stored; one call auto-generates one document live. Go/no-go at ~3:30 — lock pre-recorded fallback if shaky
4:05-4:30	Mock layer + polish	Stub doc auto-fill, compliance routing, Rocketlane export preview; tidy UI so the whole vision reads in one flow
4:30-5:00	Demo rehearsal	Run it twice, time it, pre-stage the fake project and second transcript, assign who clicks / who talks

Discipline: the conflict map is what wins; the live call is the wow. If anything slips, protect those two and mock the rest.

11. The demo (live, ~6-7 min)

Open with the wedge, not the documents. Lead with voice; the document engine is supporting.

Part 1 — The voice call (~3 min). Hand a judge a phone. The ElevenLabs agent calls them as the "sponsor" of a pre-staged project, runs a ~90-second information call, and — live — generates a piece of onboarding paperwork from what they just said. The point landed: *a call a human used to sit through just became a document, with no human on our side.*

Part 2 — The conflict map (~2-3 min). Show that a second stakeholder (pre-recorded IT lead) was also interviewed. Generate the conflict map on stage: it surfaces a **real contradiction** between what the judge just said and what the IT lead said — a timeline or scope clash the judge didn't realize they'd created. This is the "oh" moment, and it's the artifact no document tool can produce.

Part 3 — The engine + Rocketlane (~60 sec). Quickly: drop the SOW, show the Deal Profile populate, one auto-filled document, the consistency check catching a planted mismatch, then the mocked Rocketlane export. Framing line: *"All of this — the calls, the paperwork, the conflicts — lands in Rocketlane as a pre-built project. We're the input layer that gives Nitro the human context no document contains."*

Close: "The agent takes the calls that don't need a human, writes the paperwork, and tells you where your stakeholders disagree before kickoff — not three sprints in."

Why this demo wins: voice is essential and live (impossible to fake); the conflict map is genuinely novel; and the Nitro-complementary framing means we're extending the judges' product, not duplicating it.

12. Competitive landscape & why we're differentiated

Player	What they do	Why we're not them
Rocketlane / Nitro	Ingests SOWs, emails, recorded calls, configs → auto-generates plans & documents	Works on artifacts that already exist. We generate <i>net-new</i> context via outbound voice and produce the conflict map they can't. We feed them.
AI Fills (Rocketlane)	Post-meeting notetaking, doc standardization, follow-ups	Processes meetings that already happened. We <i>run</i> the meeting, by agent, and cross-check across stakeholders.
Insight7 / Tellet / Otter	Transcribe & analyze interviews someone already conducted	They analyze; we <i>conduct</i> . And none produce a cross-stakeholder conflict map.
Generic onboarding tools (HoneyBook, Content Snare, GUIDEcx, etc.)	Forms, welcome emails, doc collection, task templates	Forms get ignored and capture no tone. Voice gets done and captures the political signal.
Custom LLM SOW-parsers (Syntora-style)	Parse SOW → populate CRM/QuickBooks	Document-in, record-out. No conversation, no alignment layer.

The one-sentence differentiation: everyone else turns existing documents into records; we turn *conversations that never would have happened* into both paperwork and the alignment map that documents can't give you.

13. Why this fits Rocketlane specifically

Rocketlane's whole thesis is moving PS from "tracking work" to "executing it" via agents, and Nitro already executes the document side. The honest gap in an all-agent delivery model is the **human-context capture** at the front — the stuff that only exists in stakeholders' heads until someone talks to them. We fill exactly that gap, with voice, and hand the result to their platform. We span their named tracks — Operations (stakeholder/capacity context), Delivery (conflict/risk surfacing), Execution (calls → documents) — and our output isn't a deliverable outside their product; it's a pre-built Rocketlane project. We're the input layer their agent doesn't have.

14. Application answers (final)

What problem are you excited to solve? When a deal closes, onboarding burns weeks of human time, and a lot of it is spent on calls where the human is just a microphone — gathering requirements, collecting billing and legal details, confirming scope, scheduling. Those information calls don't need human judgment, but a person sits through each one and then writes it all up by hand: slow, inconsistent, error-prone. And because stakeholders are interviewed in isolation (if at all), projects start with hidden disagreements that don't surface until they're blockers. We want a voice agent to take the calls that don't need a human, generate the paperwork automatically, and — because it's the only thing that's spoken to everyone — surface the misalignments before kickoff.

Who's the user you have in mind — anyone you've spoken to who has faced this? Primary user is the PS / implementation PM, especially the junior-to-mid PM running several onboardings at once. Secondary users are the finance/ops people who own pieces of onboarding and the customer stakeholders who'd rather take a 10-minute call than fill out a form. From informal conversations with people in PS roles, the consistent pattern is that the "information" calls and the write-up afterward eat more time than anyone tracks, and the kickoff is always weaker than it should be because real cross-stakeholder discovery never happened.

How are you thinking about approaching it? Voice-first. We read the signed SOW once with Claude to build a structured Deal Profile — including who to call. Then an ElevenLabs voice agent runs the information calls: it interviews each stakeholder, adapts its questions live, and turns each call into structured paperwork. A second Claude pass synthesizes across every transcript to produce our signature artifact — a cross-stakeholder conflict map showing where sponsor, IT, and champion silently disagree on scope, timeline, and success — plus a kickoff packet. Underneath sits a document engine (auto-fill + consistency checks) that overlaps with Rocketlane's Nitro on purpose:

we position ourselves as the input layer that gives Nitro the human context no document contains. Lovable for the frontend and seeded dummy data; output lands in Rocketlane as a pre-built project. Everything is draft-and-flag — the human signs off and keeps the judgment calls; we take the information calls and the write-up.

What would your demo look like at the end? Fully live, led by voice. We hand a judge a phone; the agent calls them as a sponsor, runs a 90-second information call, and generates a piece of onboarding paperwork from what they said — live. Then we show a second stakeholder was interviewed and generate the conflict map on stage, surfacing a real contradiction between what the judge said and what the (pre-recorded) IT lead said. We close by dropping the SOW to show the Deal Profile populate, one auto-filled document, a consistency check catching a planted mismatch, and the mocked Rocketlane export — framed as "we're the input layer feeding Nitro." Closing line: the agent takes the calls that don't need a human, writes the paperwork, and tells you where your stakeholders disagree before kickoff.

15. Risks & mitigations (build-day)

Risk	Mitigation
Live call fails on stage	Pre-record a backup call; go/no-go decision at ~3:30
Voice integration eats time	Build the conflict map before voice; go/no-go at ~3:30; fall back to pre-recorded
Conflict map feels generic	Most iteration time goes here (2:10–3:05); pre-seed strong synthesis prompts and two planted contradictions
Judges see it as "just Nitro"	Lead with voice + conflict map; state the Nitro-complementary framing explicitly and early
Over-claiming the agent replaces all calls	Use the call-triage map — agent takes information calls, humans keep judgment calls
Extraction misses fields on a messy SOW	Use a clean prepared sample SOW; show flagging as a feature when a field is uncertain

16. Open items to lock before build day

1. **Sample SOW** — clean, realistic, EU + a triggerable condition so the matrix and conditional fills fire.
2. **Planted conflict** — script the exact contradiction (timeline is cleanest: judge-sponsor says one date, pre-recorded IT lead says another).
3. **Second-stakeholder recording** — pre-record the IT-lead call that clashes with the live judge call.
4. **Backup call** — pre-recorded sponsor call in case live voice is unstable.
5. **One call → one document** — pick the single paperwork artifact you'll generate live (a requirements summary or billing-setup sheet is cleanest).
6. **Rocketlane/Nitro export** — confirm what the integration accepts; if limited, keep it a mocked preview.
7. **ROI line** — settle the headline number you say out loud (hours/project saved × portfolio volume).